

Отчёт: Сравнение оригинальной и восстановленной из спецификаций реализации Asteroids

Дата: 29 марта 2026

Цель: Оценить консистентность, с которой AI может восстановить работающее решение из спецификаций

Оригинал: `../Asteroids` (ручная реализация)

Восстановленная версия: `./` (asteroids-specs, AI-реконструкция по GSD-спецификациям)

1. Общая статистика

Метрика	Оригинал	Восстановленная	Δ
C# файлов (runtime + editor)	68	72 (+6 editor phase scripts)	+4
Строк C# кода	~3,817	~5,506 (+750 MCP bridge)	+44%
ScriptableObject-типов	6	6	0
Префабов	12	9	-3
Пакетов (manifest.json)	44 (+ explicit modules)	15 (implicit modules)	—
Версии пакетов	—	100% идентичны по 13 общим	—
Unity версия	2022.3.x	2022.3.60f1	—
MVVM-фреймворк	com.shtl.mvvm (git)	com.shtl.mvvm (git)	идентичен

2. Архитектурное сравнение

2.1 Общая архитектура — совпадение ~70%

Обе реализации следуют **одной и той же гибридной архитектуре:**

Слой	Паттерн	Совпадает?
Модель (игровая логика)	Custom ECS-like: Entity + Component + System	Да — одинаковая иерархия
Представление (UI/визуалы)	MVVM через com.shtl.mvvm	Да — один фреймворк
Реестр сущностей	EntitiesCatalog (фабрика + привязки)	Да — та же роль
Входные данные	Unity Input System + PlayerInput-обёртка	Да — одна схема
Visitor для регистрации	IGroupVisitor + GroupCreator	Да — один паттерн
Точка входа	ApplicationEntry (MonoBehaviour) → Application → Game	Да — та же цепочка

Вывод: AI корректно воспроизвёл высокоуровневую архитектуру. Namespace `SelStrom.Asteroids`, папочная структура (`Application/`, `Model/`, `View/`, `Configs/`, `Input/`, `Utils/`), и паттерны проектирования воспроизведены верно.

2.2 Ключевые архитектурные расхождения

Аспект	Оригинал	Восстановленная
Сигнатура <code>UpdateNode</code>	<code>UpdateNode(TNode node, float dt)</code> — только компонент	<code>UpdateNode(IGameEntityModel entity, TNode node, float dt)</code> — сущность + компонент
Состав <code>System</code> -нод	Tuple из нескольких компонентов (<code>Move</code> , <code>Gun</code> , <code>ShootTo</code>)	Одиночные компоненты + type-check entity в runtime
Управление вращением	<code>Quaternion</code> + отдельный <code>ObservableValue<Vector2></code> <code>Rotation</code>	Тригонометрия <code>Atan2</code> + direction вектор из <code>MoveComponent</code>
Направление тяги	По направлению носа корабля (facing) — классический Asteroids	По текущему вектору движения — нестандартный подход
AI НЛО	Предиктивный перехват (упреждающее прицеливание)	Простое преследование (direction = target - pos)
Подсчёт боеприпасов	Обратный отсчёт (Max → 0)	Прямой отсчёт (0 → Max)
Отслеживание пула	В <code>ViewFactory</code> + <code>BaseVisual</code> с <code>Dispose()</code>	В <code>IEntityView</code> через <code>PrefabInstanceId</code>
<code>ReactiveValue</code>	Единообразно <code>ReactiveValue<T></code>	Смешанно <code>ObservableValue<T></code> / <code>ReactiveValue<T></code>

3. Послойное сравнение

3.1 Application Layer

Файл	Ориг. (строк)	Рекон. (строк)	Сходство	Ключевое отличие
Application.cs	107	265	~25%	Рекон. — полный state machine с аудио, HUD, GameOver, Leaderboard
ApplicationEntry.cs	61	106	~30%	Рекон. имеет 10 SerializeField, UGS init, RuntimeBridgeProxy
Game.cs	263	612	~20%	Рекон. добавляет волны, жизни, respawn, UFO-правила, аудио
EntitiesCatalog.cs	214	342	~40%	ViewFactory vs прямой пул; разные модели коллбэков
ModelFactory.cs	25	24	~90%	Почти идентичны
ViewFactory.cs	32	26	~60%	Разный API пула

Файл	Ориг. (строк)	Рекон. (строк)	Сходство	Ключевое отличие
IApplicationComponent.cs	10	16	~85%	Минимальные различия

Подробности по Game.cs (самое большое расхождение):

Механика	Оригинал	Восстановленная
Спавн врагов	Один таймер: случайный выбор (астероид / UFO / big UFO)	Волновая система + отдельный таймер UFO (25-40с)
Жизни	Нет — столкновение = мгновенный game over	3 жизни, +1 за 10k очков (макс. 6), respawn с неуязвимостью
Лазер	Полная реализация (Physics2D.RaycastNonAlloc)	Заглушка (<code>Debug.Log(" [Game] Laser fired")</code>)
Скоринг	<code>Model.ReceiveScore(entity)</code> — switch по типу	Прямое <code>_model.Score += data.Score</code>
UFO-правила	Нет ограничений на количество	Только один UFO; big < 10k очков, small ≥ 10k

3.2 Model Layer

Файл	Ориг.	Рекон.	Сходство	Ключевое
Model.cs	184	131	~60%	Разные коллекции (HashSet vs List), отсутствует ReceiveScore
ActionScheduler.cs	68	52	~40%	struct vs class; абсолютное vs относительное время
BaseModelSystem.cs	41	41	~75%	Разная сигнатура UpdateNode

3.3 Systems (9 файлов)

Система	Ориг.	Рекон.	Сходство	Ключевое отличие
GunSystem	28	33	~40%	Подсчёт боеприпасов инвертирован
LaserSystem	28	29	~55%	Таймер перезарядки инвертирован
LifeTimeSystem	13	17	~50%	Рекон. сам убивает entity; ориг. только декрементирует
MoveSystem	23	37	~55%	Разные алгоритмы wrapping; у ориг. есть баг
MoveToSystem	25	17	~20%	Предиктивный перехват vs простое преследование
RotateSystem	20	23	~30%	Quaternion vs тригонометрия; 90°/с vs 180°/с
ShootToSystem	27	17	~20%	Самостоятельный aim+fire vs таймер+callback
ThrustSystem	25	35	~45%	По facing vs по движению

Средневзвешенное сходство систем: ~39%

3.4 Components (9 файлов)

Компонент	Ориг.	Рекон.	Сходство
IModelComponent	7	4	~95%
GunComponent	18	20	~35%
LaserComponent	19	16	~50%
LifeTimeComponent	8	12	~65%
MoveComponent	12	12	~70%
MoveToComponent	22	10	~15%
RotateComponent	13	8	~20%
ShootToComponent	22	12	~15%
ThrustComponent	13	19	~50%

Средневзвешенное сходство компонентов: ~46%

3.5 Entities (5 файлов)

Сущность	Ориг.	Рекон.	Сходство
IGameEntityModel	9	9	~95%
IGroupVisitor	10	10	~80%
ShipModel	37	56	~45%
AsteroidModel	37	25	~50%
BulletModel	34	31	~45%
UfoBigModel/UfoModel	48	36	~45%

3.6 View Layer

Файл	Ориг.	Рекон.	Сходство	Ключевое
AsteroidVisual	23	63	~30%	«Толстые» визуалы с пулом vs тонкие биндинги
BulletVisual	29	45	~25%	Отличие в коллизиях и спрайтах
ShipVisual	38	58	~30%	7 reactive fields vs 4; rotation как float vs Vector2
UfoVisual	26	51	~25%	Collision2D vs Action (без параметров)
EffectVisual	19	45	~20%	BaseVisual vs прямой MonoBehaviour
HudVisual	35	93	~20%	Polling vs reactive bindings; разный scope
TitleScreenView	15	42	~20%	1 кнопка vs 2 кнопки + текст
BaseVisual	41	16	~15%	Полный lifecycle vs только пул-трекинг
IEntityView	10	11	~30%	Dispose() vs PrefabInstanced
BindingToExtensions	17	26	~30%	1 extension vs 3 extensions
GuiText	10	15	~40%	getter vs setter; sealed vs open

Средневзвешенное сходство View: ~26%

3.7 Screens

Файл	Ориг.	Рекон.	Сходство
AbstractScreen	13	14	~70%
TitleScreen	23	19	~40%
GameScreen	318	23	~15%

Критическое расхождение: Оригинальный `GameScreen` (318 строк) — мега-контроллер, управляющий HUD + game over + leaderboard + score submission. Восстановленная версия разбила это на 4 отдельных экрана:

- `GameScreen` (23 строки) — только HUD
- `GameOverScreen` (61 строка) — только game over
- `LeaderboardScreen` (90 строк) — только лидерборд
- Отдельные View-файлы: `GameOverView.cs`, `LeaderboardView.cs`

3.8 Leaderboard/Services

Аспект	Оригинал (6 файлов, ~340 строк)	Восстановленная (1 файл, 59 строк)
Паттерн	Interface-based (IAuthProxy, ILeaderboardProxy)	Прямые вызовы UGS SDK
Async	Корутины + CoroutineResult	async/await (Task)
Тестируемость	Высокая (mock-friendly)	Низкая (hardcoded SDK calls)
Типы данных	Собственный <code>LeaderboardEntry</code> struct	SDK-типы напрямую
Хранение имени	Metadata в записи сора	<code>UpdatePlayerNameAsync</code>

Сходство: ~15% — полностью разный подход, хотя итоговая функциональность та же.

3.9 Utilities

Файл	Ориг.	Рекон.	Сходство
GameObjectPool	87	35	~25%
GameUtils	39	19	~15%
CoroutineResult	16	28	~40%
PlayerInput	71	40	~50%

3.10 Уникальные компоненты

Только в восстановленной:

Файл	Строк	Назначение
AudioManager.cs	159	Полноценная аудиосистема (9 каналов, BeatLoop)
RuntimeBridgeProxy.cs	33	MCP-мост для рефлексии
AudioData.cs	19	ScriptableObject для аудиоклипов
Phase3-7Setup.cs	~1,918	Автоматизация создания ассетов через Editor API
GameOverScreen.cs	61	Выделенный экран game over
LeaderboardScreen.cs	90	Выделенный экран лидерборда
GameOverView.cs	111	MVVM View для game over
LeaderboardView.cs	128	MVVM View для лидерборда

Только в оригинале:

Файл	Строк	Назначение
ScoreVisual.cs	87	Комбинированный game over + leaderboard
LeaderboardEntryVisual.cs	30	MVVM-виджет строки лидерборда
LeaderboardPrefabCreator.cs	70	Editor-утилита для префаба
BulletData.cs	13	Отдельный ScriptableObject для пули
Leaderboard/ (6 файлов)	~340	Interface-based архитектура сервисов

4. Конфигурация и ассеты

Аспект	Ориг.	Рекон.	Сходство
GameData.cs	63	69	~60% — структура совпадает, рекон. добавляет defaults и AudioData
AsteroidData.cs	11	11	~90%
UfoData.cs	12	13	~80% — рекон. добавляет ShootDurationSec
GunData.cs	10	11	~95%
BaseGameEntityData.cs	8	9	~100%
Packages (manifest.json)	44 записей	15 записей	100% совпадение версий по 13 общим пакетам

5. Сводная таблица по слоям

Слой	Файлов (Ориг.)	Файлов (Рекон.)	Среднее сходство	Комментарий
Архитектура (high-level)	—	—	~70%	Паттерны и структура верны

Слой	Файлов (Ориг.)	Файлов (Рекон.)	Среднее сходство	Комментарий
Application	7	7 + 4 уникальных	~36%	Game.cs — основное расхождение
Model (Model.cs + Scheduler)	2	2	~50%	Одна парадигма, разные детали
Systems	9	9	~39%	Алгоритмы различаются существенно
Components	9	9	~46%	Поля и подходы к данным отличаются
Entities	5	5	~60%	Структура совпадает, детали расходятся
View	11	11 + 2 уникальных	~26%	Философия «толстых» vs «тонких» визуалов
Screens	3	5	~42%	Разная декомпозиция ответственности
Services/Leaderboard	6	1	~15%	Полностью разный подход
Configs	6	6	~85%	Почти идентичны
Utils	4	4	~33%	Разные алгоритмы и API

6. Итоговая оценка соответствия

По уровням

Уровень	Соответствие	Описание
Архитектурные паттерны	~70%	ECS-like + MVVM + Visitor верно воспроизведены
Структура проекта	~85%	Папки, namespaces, файловая иерархия
API поверхность	~50%	Имена классов совпадают, сигнатуры методов отличаются
Алгоритмы и логика	~35%	Физика, AI, спавн, коллизии — существенные расхождения
Стиль кода	~40%	Рекон. более «защитный» (null guards, defaults, комментарии)
Функциональность	~75%	Та же игра, но рекон. добавляет волны, жизни, аудио

Общий процент соответствия подхода и архитектуры

~48%

Расшифровка: AI верно уловил **архитектурный каркас** (паттерны, слои, зависимости, фреймворк), но значительно отклонился в **реализации конкретных алгоритмов** (физика

вращения, AI, пулинг, управление жизненным циклом). Восстановленная версия также расширила score, добавив механики (волны, жизни, аудио), которых не было в оригинале.

7. Качественный анализ ключевых расхождений

7.1 Что AI воспроизвёл верно

- **Архитектурный каркас:** ECS-like Model с Systems/Components/Entities, MVVM View, Application как фасад
- **Зависимости:** Точное совпадение пакетов и их версий
- **Конфигурационный слой:** ScriptableObject-иерархия воспроизведена почти 1:1
- **Visitor-паттерн:** IGroupVisitor + GroupCreator — тот же подход
- **Имена классов и файлов:** ~90% совпадение именования
- **Фреймворк MVVM:** Тот же com.shtl.mvvm, те же базовые классы

7.2 Что AI реализовал иначе (корректно, но по-другому)

- **Декомпозиция экранов:** 5 отдельных экранов vs 1 мега-контроллер — аргумент за рекон. (чище SRP)
- **Волновая система:** Оригинал не имеет волн — рекон. добавил из спецификаций классического Asteroids
- **Жизни/Respawn:** Оригинал — instant death; рекон. — полный lives-система
- **Аудио:** Отсутствует в оригинале, полноценно в рекон.
- **Leaderboard:** 6 файлов с interfaces vs 1 файл async/await — trade-off testability vs simplicity

7.3 Что AI реализовал некорректно / с потерей качества

- **Thrust по вектору движения** вместо facing direction — нарушает каноничную физику Asteroids
- **Отсутствие предиктивного AI** у UFO — значительное упрощение поведения
- **Лазер — заглушка** (Debug.Log) — критическая недоработка
- **Отсутствие ReceiveScore** в Model — скоринг вынесен в Game, нарушая инкапсуляцию
- **Смешение ObservableValue/ReactiveValue** — стилистическая неконсистентность

8. Оценка трудозатрат (человекочасы)

8.1 Оригинальная реализация (../Asteroids)

Компонент	Оценка (чел-ч)	Обоснование
Архитектура и проектирование	8	Custom ECS + MVVM + Visitor — нетривиальный дизайн
Model Layer (Model + Systems + Components)	12	9 систем с предиктивным AI, кватернионы, ActionScheduler
Entities	4	5 типов сущностей с composition
Application Layer (App + Game + Entry)	10	Оркестрация, input, экраны, lifecycle
EntitiesCatalog + Factories	6	Фабрики, привязки MVVM, пулинг
View Layer (11 Visuals + Bindings)	8	MVVM bindings, reactive data flow
Screens (Title + GameScreen)	8	GameScreen — сложный контроллер с states
Leaderboard Service (6 файлов)	10	Interface-based, proxies, coroutines, auth
Configs (ScriptableObjects)	3	6 типов + asset instances
Input System	3	Generated actions + wrapper
Utils (Pool, GameUtils, CoroutineResult)	4	Production pool с cleanup
Editor (LeaderboardPrefabCreator)	2	Простой скрипт
Prefabs + Scene настройка	6	Ручное создание и конфигурирование
Тестирование и отладка	8	Интеграционное тестирование, баланс
ИТОГО	92	~11.5 рабочих дней (по 8ч)

8.2 Восстановленная реализация (./asteroids-specs)

Компонент	Оценка (чел-ч)	Обоснование
Спецификации и планирование (GSD)	6	8 фаз, ROADMAP, REQUIREMENTS, STATE
Model Layer	10	Те же 9 систем, но упрощённый AI
Entities	4	5 типов + IsInvulnerable + Reset
Application Layer	14	Более сложный state machine с аудио, волнами, жизнями
EntitiesCatalog + Factories	6	Пул-трекинг, префаб-реестр
View Layer (13 Visuals)	12	«Толстые» визуалы с пулингом, null-guards, rotation
Screens (5 экранов)	8	Больше экранов, но каждый проще
AudioManager + AudioData	6	9 каналов, BeatLoop, интеграция
UgsService	2	Простой async/await wrapper
Configs	3	Почти идентичны оригиналу
Input System	2	Чуть проще (без Restart)
Utils	2	Более минимальный пул
Editor Phase Scripts (5 шт.)	12	~1,918 строк автоматизации создания ассетов
MCP Bridge	2	RuntimeBridgeProxy + McpUnityBridge

Компонент	Оценка (чел-ч)	Обоснование
Тестирование и отладка	6	Автоматизированное через GSD
ИТОГО	95	~12 рабочих дней (по 8ч)

8.3 Реальное время AI-генерации

С учётом того что восстановленная версия генерировалась AI по спецификациям:

Метрика	Оценка
Человеческое время на ревью/корректировку AI-вывода	~6-10 ч
Время AI-генерации (8 фаз GSD)	~2-4 ч (wall clock)
Эквивалент сэкономленных человекочасов	~85 ч
Коэффициент ускорения	~10x

9. Выводы

9.1 Успехи AI-реконструкции

- 1. **Архитектурная верность (70%)** — AI корректно восстановил гибридную архитектуру ECS-like + MVVM, что является сложной и нетривиальной комбинацией
- 2. **Структурная организация (85%)** — файловая иерархия, namespaces, зависимости воспроизведены точно
- 3. **Конфигурационный слой (85%)** — ScriptableObject-иерархия почти идентична
- 4. **Расширение функциональности** — AI добавил канонические механики Asteroids (волны, жизни, аудио), которые улучшают игровой опыт
- 5. **Пакетные зависимости (100%)** — точное совпадение версий

9.2 Проблемы AI-реконструкции

- 1. **Алгоритмическая точность (35%)** — физика, AI, управление состоянием существенно отличаются
- 2. **Лазер — заглушка** — критический пробел в функциональности
- 3. **Неканоничная физика тяги** — thrust по движению вместо facing нарушает игровой канон
- 4. **Упрощение AI** — потеря предиктивного перехвата UFO
- 5. **Стилистическая неконсистентность** — смешение ObservableValue/ReactiveValue, «толстые» визуалы

9.3 Общая оценка

AI способен восстановить **архитектурный каркас** с высокой точностью (~70-85%), но **конкретная реализация алгоритмов** существенно расходится (~35%). Это ожидаемо: спецификации описывают что должна делать система, но не как именно. Для продакшн-использования потребуется код-ревью и доработка алгоритмического слоя.

Рекомендация: AI-реконструкция из спецификаций — эффективный инструмент для быстрого прототипирования и восстановления проектной структуры. Для полного соответствия оригиналу необходимы дополнительные спецификации на уровне алгоритмов или прямое copy-paste ключевых формул.

10. Матрица соответствия файлов

#	Файл	Ориг. (строк)	Рекон. (строк)	Сходство
1	Application.cs	107	265	25%
2	ApplicationEntry.cs	61	106	30%
3	Game.cs	263	612	20%
4	EntitiesCatalog.cs	214	342	40%
5	ModelFactory.cs	25	24	90%
6	ViewFactory.cs	32	26	60%
7	IApplicationComponent.cs	10	16	85%
8	Model.cs	184	131	60%
9	ActionScheduler.cs	68	52	40%
10	BaseModelSystem.cs	41	41	75%
11	GunSystem.cs	28	33	40%
12	LaserSystem.cs	28	29	55%
13	LifeTimeSystem.cs	13	17	50%
14	MoveSystem.cs	23	37	55%
15	MoveToSystem.cs	25	17	20%
16	RotateSystem.cs	20	23	30%
17	ShootToSystem.cs	27	17	20%
18	ThrustSystem.cs	25	35	45%
19	GunComponent.cs	18	20	35%
20	LaserComponent.cs	19	16	50%
21	LifeTimeComponent.cs	8	12	65%
22	MoveComponent.cs	12	12	70%
23	MoveToComponent.cs	22	10	15%

#	Файл	Ориг. (строк)	Рекон. (строк)	Сходство
24	RotateComponent.cs	13	8	20%
25	ShootToComponent.cs	22	12	15%
26	ThrustComponent.cs	13	19	50%
27	IModelComponent.cs	7	4	95%
28	IGameEntityModel.cs	9	9	95%
29	IGroupVisitor.cs	10	10	80%
30	ShipModel.cs	37	56	45%
31	AsteroidModel.cs	37	25	50%
32	BulletModel.cs	34	31	45%
33	UfoBigModel.cs	48	36	45%
34	AsteroidVisual.cs	23	63	30%
35	BulletVisual.cs	29	45	25%
36	ShipVisual.cs	38	58	30%
37	UfoVisual.cs	26	51	25%
38	EffectVisual.cs	19	45	20%
39	HudVisual.cs	35	93	20%
40	TitleScreenView.cs	15	42	20%
41	BaseVisual.cs	41	16	15%
42	IEntityView.cs	10	11	30%
43	BindingToExtensions.cs	17	26	30%
44	GuiText.cs	10	15	40%
45	AbstractScreen.cs	13	14	70%
46	TitleScreen.cs	23	19	40%
47	GameScreen.cs	318	23	15%
48	GameData.cs	63	69	60%
49	AsteroidData.cs	11	11	90%
50	UfoData.cs	12	13	80%
51	GunData.cs	10	11	95%
52	BaseGameEntityData.cs	8	9	100%
53	GameObjectPool.cs	87	35	25%
54	GameUtils.cs	39	19	15%
55	CoroutineResult.cs	16	28	40%
56	PlayerInput.cs	71	40	50%
—	Среднее (56 файлов)	~3,348	~3,533	~44%